

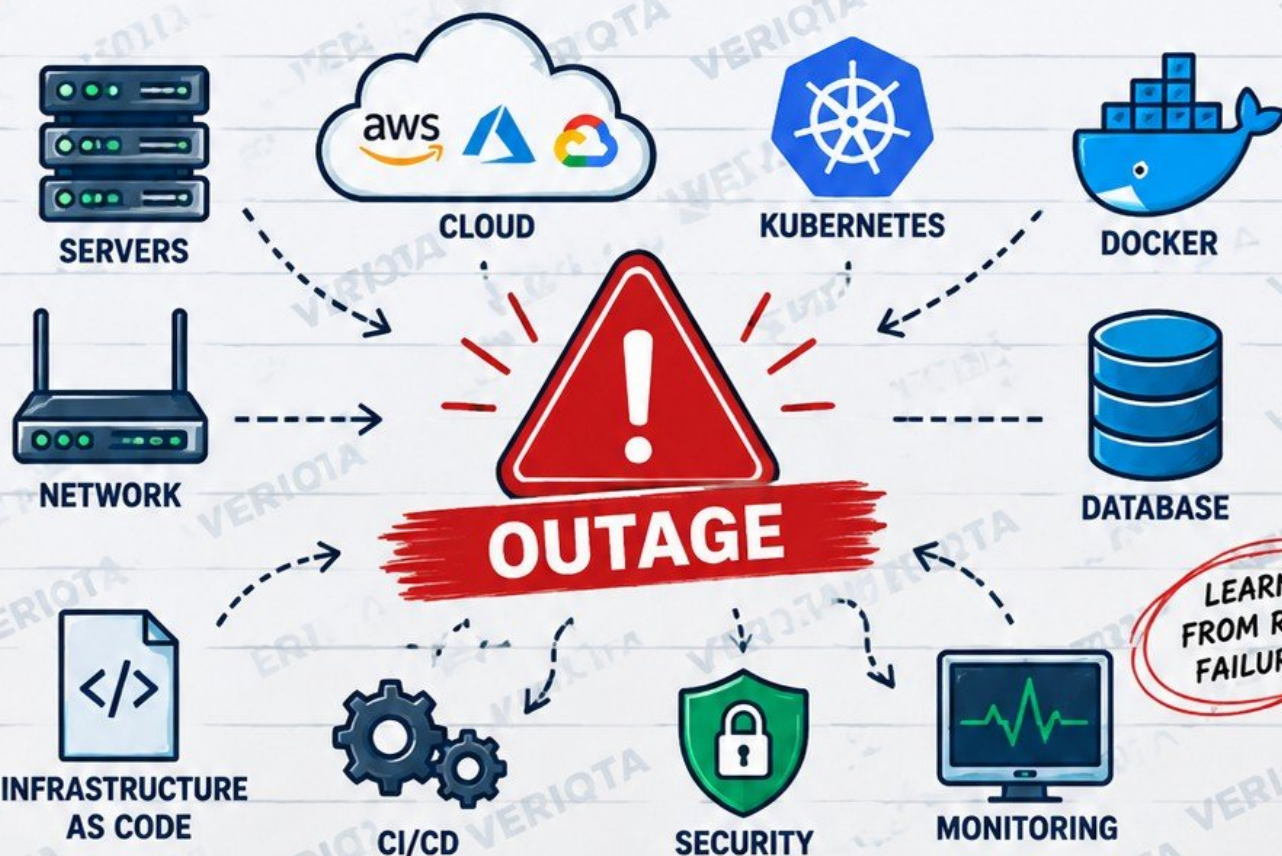
- ✓ REAL INCIDENTS
- ✓ PRACTICAL FIXES
- ✓ PRODUCTION READY
- ✓ BUILD YOUR SKILLS

# 50

## INFRASTRUCTURE OUTAGES

EVERY ENGINEER SHOULD KNOW

REAL INCIDENTS | ROOT CAUSES | DIAGNOSIS | FIXES | PREVENTION



LEARN  
FROM REAL  
FAILURES

FAIL  
LEARN  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.

VERIQTA ENGINEERING NOTEBOOK SERIES

# How Infrastructure Outages Actually Happen

Build the mental model before the incidents.

REAL INCIDENTS  
REAL LESSONS  
BIGGER YOU

## 1 What Counts as an Infrastructure Outage?

- Infrastructure is considered down or not working as expected.
- Services become unavailable, unstable or severely degraded.
- Can be in on-premises, cloud or hybrid environments.
- Affects core components like compute, storage, networking, databases or platform services.

## 2 Full Outage vs Partial Degradation

### Full Outage

- Service is completely unavailable.
- No users can access it.
- Example: entire region down.

### Partial Degradation

- Service is still running but with reduced performance.
- Some features may work.
- Example: slow API responses or intermittent errors.

## 3 Blast Radius

- The total impact of a failure across systems, services and users.
- A small problem can have a large blast radius if many services depend on it.
- Good system design limits the blast radius using isolation, redundancy and least privilege.



Goal: contain failures before they spread.

## 4 Dependency Failure

- Modern systems depend on many other services.
- A failure in one dependency can break multiple services even if your own system is working.
- Examples: DNS, database, identity provider, third-party API, cloud service, network.
- Always understand your dependencies.

## 5 Control Plane vs Data Plane

### Control Plane

- Manages, configures and controls the system.
- Example: Kubernetes API server, cloud management services.
- If it fails, you cannot make changes or manage resources.

### Data Plane

- Handles actual traffic and workloads.
- Example: application servers, containers, databases, network.
- If it fails, users cannot access services.

## 6 Infrastructure vs Application Symptoms

| Infrastructure Symptom    | Application Symptom       |
|---------------------------|---------------------------|
| Server unreachable        | App returns 500 error     |
| High CPU or memory        | Slow response time        |
| Disk full                 | Database errors           |
| Network/DNS issues        | Cannot connect to service |
| Load balancer unhealthy   | Users see downtime        |
| Kubernetes node not ready | Pods keep restarting      |

Application issues are often caused by infrastructure problems.

## 7 The Troubleshooting Loop



## 8 Junior Engineer Tip



Never start changing things before establishing what is actually failing.

- ✓ Understand the problem.
- ✓ Gather evidence.
- ✓ Avoid random changes.
- ✓ Make the smallest safe fix.
- ✓ Verify the result.

LEARN  
PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

# Linux Host Outages

## Outages 01 – 03

Common Linux infrastructure failures that can take down applications, services and entire systems. Learn the symptoms, diagnosis and fixes.



**LINUX**  
STABLE SYSTEMS  
POWER  
GREAT THINGS

### 01 CPU Saturation

- Load spikes to 100%.
- Processes become slow or unresponsive.
- The server may still be up, but everything is sluggish.
- Can be caused by runaway processes, high traffic, poorly written code or scheduled jobs.

#### DIAGNOSE IT

```
> top
> ps aux --sort=-%cpu | head
> uptime
```

#### EXAMPLE OUTPUT (top)

```
%Cpu(s): 98.7 us, 1.3 sy, 0.0 ni,
0.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
```

#### HOW TO FIX

- ✓ Identify the process using CPU.
- ✓ Stop or restart the process if safe.
- ✓ Investigate the root cause (e.g. bug, load spike, cron job).
- ✓ Consider auto-scaling or resource limits.

#### JUNIOR TIP



A high CPU server may still respond, but applications will be very slow. Always check before restarting.

### 02 Memory Exhaustion and OOM

- RAM disappears and the system becomes slow.
- Processes are killed by the Linux OOM (Out of Memory) Killer.
- Can be caused by memory leaks, high load, too many processes or insufficient RAM.
- When critical processes are killed, services and applications may fail.

#### DIAGNOSE IT

```
> free -h
> dmesg | grep -i oom
> journalctl -k | grep -i oom
```

#### EXAMPLE OUTPUT (dmesg)

```
Out of memory: Kill process 1234
(java) score 987 or sacrifice child
Killed process 1234 (java) total-vm:
2048000kB, anon-rss: 1024000kB
```

#### HOW TO FIX

- ✓ Identify which process was killed.
- ✓ Optimize the application or reduce memory usage.
- ✓ Increase RAM or add swap (if appropriate).
- ✓ Use memory limits in containers and Kubernetes.

#### COMMON MISTAKE



Ignoring OOM logs. The system may look "randomly" broken because a critical process was killed.

### 03 Disk Space Exhaustion

- Filesystem reaches 100%.
- Applications cannot write files (e.g. logs, databases, uploads).
- Services may crash or become unavailable.
- Often caused by large logs, unused files, backups or growing databases.

#### DIAGNOSE IT

```
> df -h
> du -sh /var/* | sort -h
> du -sh * | sort -h
```

#### EXAMPLE OUTPUT (df -h)

| Filesystem | Size | Used | Avail | Use% |
|------------|------|------|-------|------|
| /dev/sda1  | 50G  | 50G  | 0     | 100% |

#### HOW TO FIX

- ✓ Identify what is using space (du, logs, large files).
- ✓ Remove or archive unnecessary files.
- ✓ Rotate and compress logs.
- ✓ Increase disk size or add storage.
- ✓ Set up monitoring and alerts for disk usage.

#### PRODUCTION AWARENESS



Disk full can bring down multiple services. Monitor disk usage and set alerts before it reaches 100%.

**PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT**

**REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.**



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

LINUX  
 FAILURES  
 ARE LESSONS  
 FOR A BETTER  
 ENGINEER

# Linux Process and Filesystem Failures

## Outages 04 – 06


  
 TROUBLESHOOT  
 LEARN  
 IMPROVE  
 GROW

Real problems. Real solutions. Learn, practice, be ready.

### 04 Inode Exhaustion

Disk appears to have space but cannot create files.

#### Common Symptoms

- You cannot create new files or directories.
- Applications fail to write.
- You may see errors like: "No space left on device".
- `df -h` shows free space, but the system is still out of inodes.

#### > Diagnose It

```
> df -i
```

Check inode usage on all filesystems.

#### Example Output

| Filesystem | Inodes | IUsed  | IFree | IUse% |
|------------|--------|--------|-------|-------|
| /dev/sda1  | 262144 | 262144 | 0     | 100%  |

#### How to Fix

- ✓ Identify and remove unnecessary small files (logs, cache, temp files).
- ✓ Clean application directories.
- ✓ Consider log rotation.
- ✓ If inode usage is consistently high, review application behavior.

#### Junior Tip


 Disk space and inodes are different. You can have plenty of disk space but still run out of inodes.

### 05 Critical Service Stops

Process crashes or service becomes inactive.

#### Common Symptoms

- Application or system service is not running.
- Users cannot access the service.
- Service keeps stopping or restarting.
- May be caused by configuration errors, resource issues or dependency failures.

#### > Diagnose It

```
> systemctl status <service>
> journalctl -u <service>
```

#### Example Output

```
• example.service - Example Service
  Loaded: loaded (/etc/systemd/...)
  Active: inactive (dead)
  ...
journalctl -u example.service
... error: failed to start ...
```

#### How to Fix

- ✓ Check logs to find the root cause.
- ✓ Fix configuration or environment issues.
- ✓ Ensure dependencies are running.
- ✓ Restart the service after fixing the issue: `systemctl restart <service>`.

#### Common Mistake


 Just restarting the service without checking the logs. The service will likely fail again.

### 06 Filesystem Becomes Read-Only

Writes suddenly fail due to filesystem or underlying storage problems.

#### Common Symptoms

- Cannot create, modify or delete files.
- Applications fail to write data.
- You may see errors like: "Read-only file system".
- Often caused by filesystem corruption, storage issues, or the kernel remounting the filesystem as read-only for safety (e.g. after I/O errors).

#### > Diagnose It

```
> mount | grep 'on / '
> dmesg | tail
```

Check if the filesystem is mounted as read-only and look for related kernel messages.

#### Example Output

```
/dev/sda1 on / type ext4 (ro,relatime,...)
[ 1234.567] EXT4-fs error (device sda1):
I/O error, remounting filesystem read-only
```

#### How to Fix

- ✓ Investigate kernel logs (dmesg).
- ✓ Check storage health (disk errors, I/O issues).
- ✓ Do not force remount as read-write until you understand the cause.
- ✓ Fix underlying storage or filesystem issues, then remount if safe.

#### Production Awareness


 A read-only filesystem is a warning sign. It usually means there is a serious underlying problem. Investigate the root cause before attempting recovery.

 PRACTICE  
 TROUBLESHOOT  
 IMPROVE  
 REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

REAL  
PROBLEMS  
REAL SOLUTIONS  
BETTER  
ENGINEERS

# Network Connectivity Outages

## Outages 07 - 09

  
NETWORKS  
CONNECT  
OPPORTUNITIES  
KEEP LEARNING

Understand network failures. Diagnose faster. Keep systems running.

### 07 DNS Resolution Failure

Hostnames stop resolving or return the wrong IP address.

#### Common Symptoms

- Cannot resolve domain names (e.g. example.com).
- Slow or intermittent resolution.
- Applications cannot reach external services.
- May see errors like: "Temporary failure in name resolution".
- Internal services may also fail if DNS is down.

#### Diagnose It

```
> dig example.com
> nslookup example.com
> cat /etc/resolv.conf
```

#### Example Output (dig)

```
;; connection timed out;
;; no servers could be reached
```

#### How to Fix

- ✓ Check your DNS server configuration in /etc/resolv.conf.
- ✓ Test with a public DNS server (e.g. 8.8.8.8).
- ✓ Restart the DNS service if you manage it.
- ✓ Check network connectivity to the DNS server.

#### Junior Tip

 Try resolving both external (e.g. google.com) and internal domains to identify the scope.

### 08 Routing Failure

Correct host exists but traffic cannot reach it.

#### Common Symptoms

- Host is reachable on the network but packets do not arrive.
- traceroute shows traffic stopping at a hop.
- May work for some networks but fail for others.
- Caused by missing or incorrect routes.
- Can be due to on-prem, cloud or hybrid network issues.

#### Diagnose It

```
> ip route
> traceroute <host>
> tracepath <host>
```

#### Example Output (traceroute)

```
traceroute to 10.0.1.10 (10.0.1.10)
 1  10.0.0.1  1.234 ms
 2  * * *
 3  * * *
```

#### How to Fix

- ✓ Check routing table (ip route).
- ✓ Verify next hop and gateway.
- ✓ Fix missing or incorrect routes.
- ✓ Check network devices (routers, VPN, transit gateway).
- ✓ Confirm route propagation in cloud environments.

#### Common Mistake

 Assuming the host is down when the real issue is a missing or incorrect route.

### 09 Firewall Blocks Production Traffic

Service is running but clients cannot connect.

#### Common Symptoms

- Service is healthy but unreachable.
- Connections timeout or are refused from certain IPs or networks.
- Only some clients can connect (e.g. internal works, external fails).
- Caused by firewall rules, security groups or network ACLs.
- Ports, protocols or source/destination rules are blocking traffic.

#### Diagnose It

```
> ss -ltnp
> nc -zv <host> <port>
> sudo iptables -L (or firewall-cmd)
```

#### Example Output (nc)

```
nc: connect to 203.0.113.10 port 80
(tcp) timed out: Operation timed out
```

#### How to Fix

- ✓ Verify firewall rules allow the traffic (source, destination, port, protocol).
- ✓ Check security groups and network ACLs (cloud environments).
- ✓ Confirm the service is listening on the expected port.
- ✓ Test from the client network after changes.
- ✓ Follow the principle of least privilege.

#### Production Awareness

 Firewall changes can affect multiple services. Document rules, test in a safe environment and monitor after changes.

PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

**PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT**

# Network Service Failures

## Outages 10 - 12

**GOOD  
NETWORKS  
KEEP  
BUSINESSES  
RUNNING**

Common network service issues that stop applications from working.  
Learn the symptoms, how to diagnose and how to fix them.

### 10 Port Already in Use

Application cannot bind to a port because it is already in use.

#### Common Symptoms

- Application fails to start.
- Error message: "Address already in use".
- Another process is already listening on the port.
- Often happens after a previous instance did not shut down properly.

#### > Diagnose It

```
> ss -lntp | grep :<port>
> lsof -i :<port>
```

#### Example Output (ss)

```
LISTEN 0      128 0.0.0.0:8080
users:(("java",pid=1234,fd=7))
```

#### How to Fix

- Identify the process using the port.
- Stop the process if safe.
- Change the application port if needed.
- Ensure proper shutdown on deployment.

#### Junior Tip

A port can only be used by one process at a time on the same IP address.

### 11 Connection Refused

Host is reachable, but the service is not accepting connections.

#### Common Symptoms

- You can reach the host (ping works).
- Connection fails immediately.
- Error message: "Connection refused".
- Service is not running, crashed or not listening on the expected port.
- Could also be blocked by a local firewall.

#### > Diagnose It

```
> ss -lntp
> nc -zv <host> <port>
> systemctl status <service>
```

#### Example Output (nc)

```
nc: connect to 10.0.1.10 port 80
(tcp) failed: Connection refused
```

#### How to Fix

- Check if the service is running.
- Verify the correct port is configured.
- Check firewall rules.
- Review application logs.
- Restart the service if needed.

#### Common Mistake

Assuming it is a network problem when the service is simply not running.

### 12 Connection Timeout

Packets disappear or the destination never responds.

#### Common Symptoms

- The host may be unreachable or not responding.
- Requests hang and eventually time out.
- Error message: "Connection timed out".
- Can be caused by network issues, firewall rules, routing problems or a down host.

#### > Diagnose It

```
> ping <host>
> nc -zv <host> <port>
> traceroute <host>
> tracepath <host>
```

#### Example Output (nc)

```
nc: connect to 10.0.1.10 port 80
(tcp) timed out: Operation timed out
```

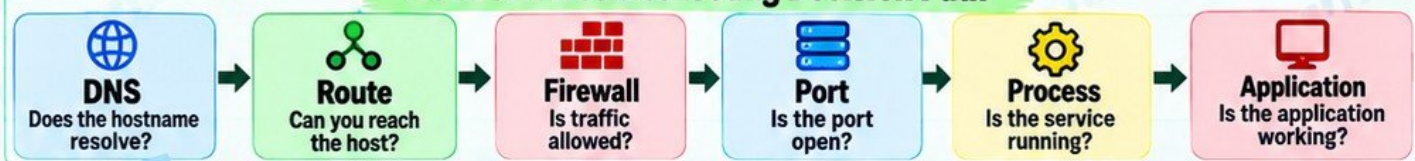
#### How to Fix

- Check network connectivity and routes.
- Verify firewall and security group rules.
- Confirm the host is up.
- Check for network ACLs or ISP issues.
- Trace the path to find where it fails.

#### Production Awareness

A timeout often means the request is not reaching the service. Check every point in the network path.

## Network Troubleshooting Decision Path



**REAL  
INCIDENTS  
REAL LESSONS  
STRONGER YOU**

**REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.**

REAL  
INFRASTRUCTURE  
REAL INCIDENTS  
REAL SOLUTIONS  
VERIQTA

# Load Balancer and Traffic Outages

## Outages 13 - 15

OBSERVE  
INVESTIGATE  
FIX  
VERIFY  
LEARN  
IMPROVE

A load balancer can make a healthy application look completely unavailable. Understand common failures, how to diagnose them and how to fix them.

### 13 Load Balancer Has No Healthy Targets

All backend targets are unhealthy, so the load balancer stops sending traffic to them.

#### Common Symptoms

- Clients get 503 (Service Unavailable) or 502 (Bad Gateway).
- Load balancer shows 0 healthy targets.
- Application is running but not receiving traffic.
- Can be caused by failing health checks, crashed instances or network issues.

#### Diagnose It

- > `aws elbv2 describe-target-health`
- > `aws elbv2 describe-target-groups`
- > Check target instance status

#### How to Fix

- ✓ Check why targets are unhealthy.
- ✓ Fix the underlying issue (instance, container, app).
- ✓ Verify health check endpoint works.
- ✓ Confirm targets return the expected status code (e.g. 200).

#### Junior Tip



A healthy application can still be down if the load balancer sees no healthy targets.

### 14 Health Checks Are Misconfigured

The load balancer marks healthy targets as unhealthy because the health check is wrong.

#### Common Symptoms

- Targets show as unhealthy in the load balancer.
- Application works when accessed directly but fails through the load balancer.
- Wrong path, port, protocol or expected status code.
- Health check times out or receives an unexpected response.

#### Diagnose It

- > Check health check path, port, protocol
- > Test endpoint manually (curl)
- > Review load balancer health check details in console or CLI

#### How to Fix

- ✓ Use the correct path (e.g. /health).
- ✓ Match the correct port and protocol (HTTP/HTTPS).
- ✓ Set the right expected status code (e.g. 200).
- ✓ Increase timeout if the application is slow to respond.

#### Junior Tip



Your application may be healthy but returning 301, 302 or 404, which makes the health check fail.

### 15 Traffic Is Routed to the Wrong Backend

The load balancer sends traffic to the wrong targets due to misconfiguration, wrong rules or DNS issues.

#### Common Symptoms

- Users reach the wrong application or environment (e.g. staging instead of production).
- Some requests work, some fail.
- Path or host-based rules are incorrect.
- DNS points to the wrong load balancer.
- Can happen after an infrastructure change or deployment.

#### Diagnose It

- > Review listener rules and target groups
- > Check DNS records and domains
- > Verify which targets are receiving traffic
- > Use `curl -v` to check the response

#### How to Fix

- ✓ Correct listener rules (host, path, priority).
- ✓ Update DNS to the right load balancer.
- ✓ Verify target group configuration.
- ✓ Test end-to-end after changes.

#### Junior Tip



Always verify you are pointing to the correct environment. A small DNS or rule change can send all traffic to the wrong place.

## How a Load Balancer Handles Traffic



Even if your application is running, users will see it as down if the load balancer cannot reach or mark the targets as healthy.

PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

REAL  
CLOUD  
INCIDENTS  
REAL  
ENGINEERS  
VERIQTA

# Cloud Compute Outages

## Outages 16 – 18

CLOUD  
SERVERS POWER  
OPPORTUNITIES  
KEEP THEM  
RUNNING

Understand cloud compute failures. Diagnose faster. Build more resilient systems.

### 16 VM or Instance Becomes Unreachable

You cannot connect to the instance (SSH, RDP or console).

#### Common Symptoms

- SSH connection timeout.
- Host is unreachable (no ping).
- Instance appears "running" but not accessible.
- Security group or network blocks access.
- Instance may be stuck during boot or OS not responding.

#### Diagnose It

```
> ping <instance-ip>
> ssh -v user@<instance-ip>
> telnet <ip> <port>
> Check security group and route tables
```

#### How to Fix

- Verify instance status in the cloud console.
- Check security groups, NACLs and routes.
- Confirm correct IP and key/credentials.
- Use instance console for boot issues.
- Reboot or replace the instance if needed.

#### Junior Tip

If the instance shows as running but you still cannot connect, always check security groups and network routes first.

#### KEY CONCEPTS TO COVER

- Instance health and status checks
- SSH connectivity and access
- Security groups and network rules
- Boot failures and recovery
- User data and initialization scripts
- Immutable infrastructure
- Why relying on one server creates risk

### 17 Instance Runs Out of CPU or Memory

The instance is running but becomes slow or unresponsive due to resource exhaustion.

#### Common Symptoms

- High CPU usage (near 100%).
- High memory usage.
- Applications become slow or unresponsive.
- OOM (Out of Memory) kills processes.
- May affect multiple services on the instance.

#### Diagnose It

```
> top / htop
> free -h
> vmstat 1 5
> dmesg | grep -i oom
```

#### How to Fix

- Identify the resource-hungry process.
- Optimize the application.
- Increase instance size (scale up).
- Add auto-scaling.
- Set resource limits and monitoring.

#### Junior Tip

High CPU or memory can make the instance look "down". Check resource metrics before assuming it has failed.

### 18 Instance Replacement Breaks the Service

The instance is replaced or recreated but the service stops working.

#### Common Symptoms

- New instance is running but application fails.
- User data did not run correctly.
- Configuration or environment is missing.
- Dependencies (databases, storage, secrets) not reattached.
- Service works on old instance but not on the new one.

#### Diagnose It

```
> Check instance user data logs
> Compare configuration
> Check attached volumes and IAM roles
> Review application and service logs
```

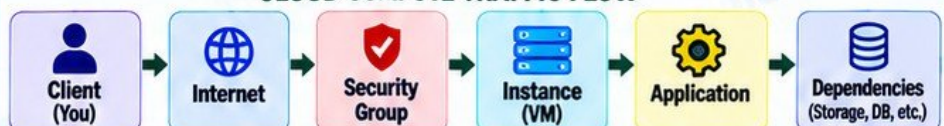
#### How to Fix

- Verify user data and startup scripts.
- Ensure all dependencies are attached.
- Use infrastructure as code for consistent builds.
- Test the new instance before putting it in production.

#### Junior Tip

Use immutable infrastructure. Always deploy from code, not manual changes.

#### CLOUD COMPUTE TRAFFIC FLOW



Even if the application is healthy, network, security or configuration issues can make it appear completely unavailable.

PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.

REAL  
CLOUD  
INCIDENTS  
REAL  
SOLUTIONS  
VERIQTA

# Cloud Network Outages

## Outages 19 - 21

UNDERSTAND  
TROUBLESHOOT  
SOLVE  
IMPROVE  
GROW

Trace the packet. Find the issue. Fix with confidence. No guessing.

### 19 Security Group Blocks Required Traffic

Traffic is blocked by security group rules, even though the instance and application are running.

#### Common Symptoms

- Cannot connect to the instance (SSH, RDP or application port).
- Connection timeout.
- Works from some IPs but not others.
- Application is healthy but not reachable.
- Inbound or outbound traffic is blocked.

#### Diagnose It

- > `aws ec2 describe-security-groups`
- > Check inbound and outbound rules
- > Test with `nc -zv <host> <port>`
- > Verify source/destination IP

#### How to Fix

- ✓ Add the required inbound/outbound rule.
- ✓ Use the correct source IP or CIDR range.
- ✓ Follow the principle of least privilege.
- ✓ Test the connection after the change.

#### Junior Tip

Security groups are stateful. If you can't connect, check both inbound and outbound rules.

### 20 Route Table Breaks Connectivity

The route table is missing or has an incorrect route, so traffic cannot reach the destination.

#### Common Symptoms

- Instance cannot reach the internet or another network.
- Traffic is routed to the wrong target (e.g. blackhole).
- VPC peering or VPN not working.
- Some subnets work, others don't.
- Traceroute shows traffic stopping at a specific hop.

#### Diagnose It

- > `aws ec2 describe-route-tables`
- > Check for correct route (destination)
- > Look for blackhole or missing routes
- > Use `traceroute` or `tracpath`

#### How to Fix

- ✓ Add or correct the missing route.
- ✓ Ensure the next hop (IGW, NAT, VPN, peering) is correct.
- ✓ Associate the right route table with the subnet.
- ✓ Verify end-to-end connectivity.

#### Junior Tip

Always check which route table is associated with the subnet. The correct route in the wrong table does nothing.

### 21 NAT or Internet Gateway Failure/Misconfiguration

Instances in private subnets cannot reach the internet due to a NAT gateway or internet gateway issue.

#### Common Symptoms

- Instances in private subnets cannot access the internet.
- Software updates fail.
- Cannot reach external services (e.g. APIs, package repositories).
- NAT gateway is deleted, misconfigured or has no route.
- Internet gateway is not attached or not in the route table.

#### Diagnose It

- > Check NAT/IGW status in console
- > `aws ec2 describe-nat-gateways`
- > `aws ec2 describe-internet-gateways`
- > Verify route table and subnet association

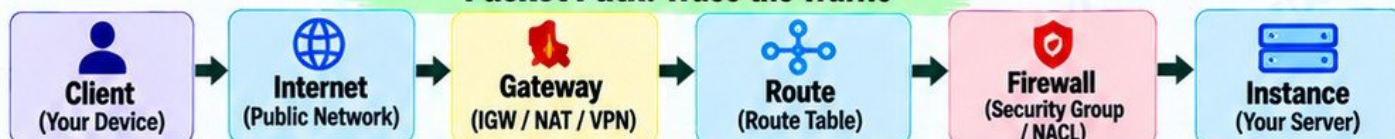
#### How to Fix

- ✓ Ensure the NAT gateway or IGW is available and attached.
- ✓ Check route table has 0.0.0.0/0 to NAT or IGW.
- ✓ Verify network ACLs and security groups.
- ✓ Test outbound connectivity (`curl`, `ping`).

#### Junior Tip

Private instances need a NAT gateway (or a proxy) to reach the internet. No NAT = no internet.

### Packet Path: Trace the Traffic



Trace each step. Identify where the packet stops. Fix the root cause.

PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

REAL INCIDENTS  
REAL SOLUTIONS  
REAL ENGINEERS  
VERIQTA

# Storage Outages

## Outages 22 - 24

STORAGE  
RELIABILITY  
KEEPS  
APPLICATIONS  
RUNNING

Storage issues can bring down applications. Understand the causes, diagnose quickly and fix with confidence.

### 22 Storage Volume Fills Up

The storage volume reaches 100% capacity, causing applications, logs or databases to fail.

#### Common Symptoms

- Disk space at or near 100%.
- Applications cannot write files.
- Logs stop being written.
- Databases become read-only or stop working.
- System becomes slow or unresponsive.
- Errors like "No space left on device".

#### Diagnose It

```
> df -h
> du -sh /path
> lsblk
> Check application and database logs
```

#### How to Fix

- ✓ Identify what is using the space.
- ✓ Clean up old logs or unused files.
- ✓ Increase the volume size (if supported).
- ✓ Move data to a larger or separate volume.
- ✓ Set up monitoring and alerts for low disk space.

#### Junior Tip

Don't just delete random files. Understand what is safe to remove and consider long-term capacity planning.

### 23 Storage Volume Is Detached or Unavailable

The storage volume is detached, not attached correctly, or becomes unavailable due to an infrastructure issue.

#### Common Symptoms

- Instance cannot find the volume.
- Filesystem is missing or not mounted.
- Application fails to start or loses data.
- Errors like "device not found" or "no such file or directory".
- Volume shows as "in-use" or "available" but not attached correctly.

#### Diagnose It

```
> lsblk
> blkid
> mount | grep /dev
> Check cloud console (e.g. AWS EC2)
```

#### How to Fix

- ✓ Check the volume status in the cloud console.
- ✓ Attach the volume to the correct instance.
- ✓ Verify the filesystem and mount.
- ✓ Update /etc/fstab if needed.
- ✓ Check for multi-attach or zone issues (e.g. EBS in AWS).

#### Junior Tip

Always verify the volume is in the same availability zone and attached to the correct instance.

### 24 Storage Latency or IOPS Saturation

The storage volume becomes slow due to high latency or reaching its IOPS/throughput limits.

#### Common Symptoms

- Applications become slow.
- High read/write latency.
- Database queries take longer than normal.
- CPU is low but disk wait is high.
- IOPS or throughput limits are reached.
- Timeouts and performance errors.

#### Diagnose It

```
> iostat -x 1
> iotop
> dstat -d
> Check cloud monitoring metrics (e.g. CloudWatch, Azure Monitor)
```

#### How to Fix

- ✓ Check current IOPS/throughput vs provisioned limits.
- ✓ Increase volume size or provisioned IOPS/throughput.
- ✓ Use a higher performance storage type (e.g. gp3, io2, io2 Block Express).
- ✓ Optimize application queries and access patterns.
- ✓ Distribute load across multiple volumes if needed.

#### Junior Tip

High latency is not always a network issue. Check storage metrics before blaming the application.

#### KEY CONCEPTS TO REMEMBER

- ✓ Capacity (size and free space)
- ✓ Mounts (correctly attached and mounted)
- ✓ Latency (response time)
- ✓ Throughput (data transfer rate)
- ✓ IOPS (input/output operations per second)
- ✓ Persistent storage (retains data)
- ✓ Monitoring thresholds (set alerts early)

#### Storage in the Application Stack



When storage fails, everything that depends on it can fail.  
Monitor, plan and scale before you hit the limit.

PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.

REAL INCIDENTS  
REAL SOLUTIONS  
REAL ENGINEERS  
VERIQTA

# Database Infrastructure Outages

## Outages 25 - 27

  
STABLE DATABASES  
POWER GREAT APPLICATIONS

From infrastructure issues to real user impact. Understand, diagnose and fix database outages.

### 25 Database Connection Limit Exhaustion

The database reaches the maximum number of allowed connections, so new connections are rejected.

#### Common Symptoms

- Application errors: "too many connections" or "connection refused".
- Users see timeouts or 5xx errors.
- Existing connections may still work.
- Applications become slow or unresponsive.
- Connection pool keeps retrying and makes it worse.

#### Diagnose It

```
> SHOW STATUS LIKE 'Threads_connected';
> SHOW VARIABLES LIKE 'max_connections';
> SELECT count(*) FROM information_schema.processlist;
```

#### How to Fix

- ✓ Identify and close idle connections.
- ✓ Increase connection limits (if safe).
- ✓ Use and configure a connection pool.
- ✓ Fix connection leaks in the application.
- ✓ Monitor connection usage.

#### Junior Tip

 A connection pool limits and reuses connections. Without it, high traffic can easily exhaust the database.

#### KEY CONCEPTS TO REMEMBER

- ✓ Connection limits and pooling
- ✓ Storage capacity and growth
- ✓ High availability and failover
- ✓ Monitoring and alerting
- ✓ Backups and recovery
- ✓ Application retry logic
- ✓ Infrastructure symptoms → Application impact

### 26 Database Storage Exhaustion

The database storage reaches its limit, causing write failures and service disruption.

#### Common Symptoms

- Write operations fail (INSERT, UPDATE, DELETE).
- Error messages: "no space left on device" or "disk full".
- Database becomes read-only (some systems).
- Backups may also fail.
- Application errors and increased latency.

#### Diagnose It

```
> SHOW TABLESPACE;
> SELECT table_schema,
SUM(data_length + index_length)/
1024/1024/1024 AS size_gb
FROM information_schema.tables
GROUP BY table_schema;
```

#### How to Fix

- ✓ Identify large tables and data growth sources.
- ✓ Clean up old or unnecessary data.
- ✓ Increase storage (scale up).
- ✓ Enable storage monitoring and alerts.
- ✓ Implement data retention and archiving.

#### Junior Tip

 Storage fills up over time. Set alerts before it reaches the limit.

### 27 Database Failover Causes Service Disruption

During a failover (planned or unplanned), the database endpoint changes or becomes temporarily unavailable, causing application errors.

#### Common Symptoms

- Sudden loss of database connectivity.
- Increased error rate and latency.
- Application timeouts.
- Endpoint DNS may still point to the old instance (DNS TTL).
- Read/write operations fail during failover.
- Applications without retry logic fail.

#### Diagnose It

```
> Check database status in cloud console (e.g. AWS RDS, Azure, GCP).
> Check DNS records and endpoint.
> Review failover events in logs.
> Test connectivity and latency.
```

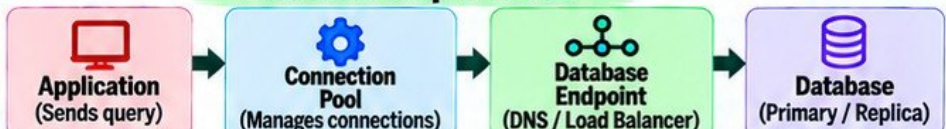
#### How to Fix

- ✓ Wait for failover to complete (if in progress).
- ✓ Ensure applications use the correct endpoint (not hardcoded IP).
- ✓ Use retry logic and connection pooling.
- ✓ Validate DNS TTL and failover settings.
- ✓ Test failover regularly.

#### Junior Tip

 Always use the database endpoint provided by your cloud provider. Do not hardcode IP addresses.

### Database Request Flow



When the database is unavailable, users see errors, timeouts or a slow application.  
Always connect the infrastructure issue to the real user experience.

PRACTICE TROUBLESHOOT  
IMPROVE REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

REAL INCIDENTS  
REAL SOLUTIONS  
REAL ENGINEERS  
VERIQTA

# Docker Infrastructure Outages

## Outages 28 – 30

CONTAINERS  
POWER  
MODERN  
INFRASTRUCTURE  
KEEP THEM  
RUNNING

Understand Docker failures. Diagnose faster. Keep your applications running.

### 28 Container Repeatedly Exits

The container starts but stops immediately or keeps restarting (crash loop).

#### Common Symptoms

- Container exits with a non-zero status code.
- CrashLoopBackOff (in orchestration).
- Application error in logs.
- Misconfiguration (environment variables, file paths, permissions).
- Health check failures.
- Container keeps restarting.

#### Diagnose It

```
> docker ps -a
> docker logs <container>
> docker inspect <container>
> Check exit code and restart count
```

#### How to Fix

- ✓ Read container logs and identify the root cause.
- ✓ Fix configuration, environment variables or missing files.
- ✓ Test the container locally before deploying.
- ✓ Update the image and redeploy.

#### Junior Tip

A container exit code tells you why it stopped. Use docker logs and docker inspect to find the reason.

### 29 Container Cannot Reach a Dependency

The container is running but cannot connect to a database, API or another service.

#### Common Symptoms

- Connection timeout or refused.
- DNS resolution failure.
- Cannot reach another container or external service.
- Wrong network configuration.
- Service name or hostname not resolving.
- Application works locally but fails in container.

#### Diagnose It

```
> docker exec -it <container> sh
> ping <service>
> nslookup <service>
> curl <service>:<port>
> docker network inspect <network>
```

#### How to Fix

- ✓ Verify the container is on the correct network.
- ✓ Use correct service names (Docker DNS).
- ✓ Check network policies and firewalls.
- ✓ Confirm the dependency is running and reachable.
- ✓ Test connectivity from inside the container.

#### Junior Tip

Use docker exec to test network connectivity from inside the container, not just from the host.

### 30 Host Resource Exhaustion Takes Down Containers

The host runs out of CPU, memory or disk space, causing containers to become slow, unstable or stop.

#### Common Symptoms

- Multiple containers become slow or unresponsive.
- Containers are killed (OOM).
- High CPU or memory usage on the host.
- Disk space is full.
- New containers cannot start.
- Affects many containers, not just one.

#### Diagnose It

```
> docker stats
> top / htop (on host)
> free -h (on host)
> df -h (on host)
> dmesg | grep -i oom
```

#### How to Fix

- ✓ Identify which resource is exhausted (CPU, memory, disk).
- ✓ Clean up unused containers, images and volumes.
- ✓ Increase host resources or set limits and reservations.
- ✓ Use monitoring and alerts.
- ✓ Plan capacity based on real usage.

#### Junior Tip

If many containers are affected, it's usually a host problem, not a container problem. Check host resources first.

### Container vs Host Failure

#### Container Failure

- ✓ Only one container is affected
- ✓ Application or configuration issue
- ✓ Check container logs and config
- ✓ Fix and redeploy

#### Host Failure

- ✓ Multiple containers affected
- ✓ High CPU, memory or disk usage
- ✓ Check host metrics and system logs
- ✓ Clean up, increase resources, or migrate

### Docker Troubleshooting Workflow



Identify the scope. Is it one container or the whole host?  
This will help you find the root cause faster.

PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

REAL INCIDENTS  
REAL SOLUTIONS  
REAL ENGINEERS  
VERIQTA

# Kubernetes Workload Outages

## Outages 31 – 33

KUBERNETES  
POWERS  
MODERN  
APPLICATIONS  
KEEP THEM  
RUNNING

Diagnose Kubernetes workload issues systematically. Inspect Events. Fix the real cause.

### 31 CrashLoopBackOff

The container starts but keeps crashing, so Kubernetes restarts it repeatedly.

#### Common Symptoms

- Pod status shows CrashLoopBackOff.
- Container exits with a non-zero exit code.
- Application errors in logs.
- Misconfiguration (env variables, config files, secrets).
- Dependency is not ready.
- Liveness probe failures.

#### Diagnose It

```
> kubectl get pods
> kubectl describe pod <pod-name>
> kubectl logs <pod-name>
> kubectl logs <pod-name> --previous
> Check the Events section
```

#### How to Fix

- Check container logs for the root cause.
- Verify configuration, environment variables and secrets.
- Ensure dependent services are ready.
- Fix application errors and rebuild image if needed.
- Adjust probes and resource limits.

#### Junior Tip

Don't keep deleting the Pod. Read the logs and Events to find the real reason.

#### KEY CONCEPTS TO REMEMBER

- Pod status and lifecycle
- Read and understand Events
- Use kubectl describe for detailed information
- Check logs before making changes
- Common causes: image, configuration, resources, network, storage, probes
- Fix the root cause, don't just delete and recreate

### 32 ImagePullBackOff

Kubernetes cannot pull the container image from the registry, so the Pod cannot start.

#### Common Symptoms

- Pod status shows ImagePullBackOff or ErrImagePull.
- Image not found (404).
- Authentication/authorization failure.
- Wrong image name or tag.
- Network/DNS issues to the registry.
- Rate limit errors (e.g. Docker Hub).

#### Diagnose It

```
> kubectl get pods
> kubectl describe pod <pod-name>
> Check the Events section
> Verify image name and tag
> Test registry access and credentials
```

#### How to Fix

- Correct the image name and tag.
- Ensure the image exists in the registry.
- Configure image pull secrets (if private registry).
- Verify network access and DNS resolution to the registry.
- Check for rate limits and authentication issues.

#### Junior Tip

The Events section usually tells you exactly why the image pull failed. Read it before making changes.

### 33 Pods Remain Pending

The Pod is created but stays in Pending state because it cannot be scheduled on a node.

#### Common Symptoms

- Pod status shows Pending.
- Insufficient CPU or memory.
- No available nodes.
- Node selector, taints or tolerations mismatch.
- PersistentVolumeClaim (PVC) not bound.
- Image pull secrets missing.

#### Diagnose It

```
> kubectl get pods
> kubectl describe pod <pod-name>
> Check the Events section
> kubectl get nodes
> kubectl get pvc
```

#### How to Fix

- Check node resources (CPU, memory).
- Verify node selectors, taints and tolerations.
- Ensure the required PVC is bound and storage is available.
- Check quotas and limits in the namespace.
- Fix image pull secrets or other missing dependencies.

#### Junior Tip

Pending means the Pod is waiting for something. Always check the Events section for the exact reason.

#### Kubernetes Troubleshooting Flow



Inspect. Understand. Fix. Don't guess. Don't just delete.

PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

REAL INCIDENTS  
REAL SOLUTIONS  
REAL ENGINEERS  
VERIQTA

# Kubernetes Networking Outages

## Outages 34 – 36

  
**NETWORKING  
CONNECTS  
YOUR  
APPLICATIONS  
KEEP IT  
RUNNING**

Trace the traffic. Test each layer. Find the real issue. Fix with confidence.

### 34 Kubernetes Service Cannot Reach Pods

The Service exists but cannot reach the backend Pods, so traffic is not forwarded.

#### Common Symptoms

- Service has no or incorrect endpoints.
- Endpoints show <none>.
- Pod labels do not match selector.
- Pods are not ready.
- Network policies are blocking traffic.
- Application works inside the Pod but not through the Service.

#### Diagnose It

```
> kubectl get svc
> kubectl get endpoints
> kubectl describe svc <service-name>
> kubectl get pods -l <label>
> kubectl describe pod <pod-name>
```

#### How to Fix

- ✓ Verify Service selector matches Pod labels.
- ✓ Ensure Pods are in Ready state.
- ✓ Check endpoints list.
- ✓ Review network policies.
- ✓ Test connectivity from within the cluster (using a temporary debug Pod).

#### Junior Tip

If endpoints are <none>, the Service has no backend Pods. Check labels and Pod readiness instead of recreating the Service.

### 35 Ingress Stops Routing Traffic

The Ingress is not routing external traffic to the Service, so users cannot reach the application.

#### Common Symptoms

- 404 errors or default backend page.
- Ingress rules or host/path mismatch.
- TLS/SSL certificate issues.
- Ingress controller not running.
- Service is healthy but not receiving traffic.
- Changes to Ingress not applied.

#### Diagnose It

```
> kubectl get ingress
> kubectl describe ingress <name>
> kubectl get pods -n <ingress-ns>
> kubectl logs <ingress-controller-pod>
> kubectl get svc (check backend service)
```

#### How to Fix

- ✓ Verify host and path configuration.
- ✓ Check TLS certificates and secrets.
- ✓ Ensure ingress controller is running.
- ✓ Confirm the backend Service and endpoints.
- ✓ Review ingress controller logs.

#### Junior Tip

Test the backend Service directly. If the Service works but Ingress does not, the problem is in the Ingress configuration or controller.

### 36 Cluster DNS Failure

DNS resolution inside the cluster is not working, so Pods cannot resolve service names.

#### Common Symptoms

- Pods cannot resolve service names (e.g. "no such host").
- Applications fail to connect to other services.
- nslookup or dig fails inside Pods.
- CoreDNS Pods are not running.
- DNS works intermittently.

#### Diagnose It

```
> kubectl get pods -n kube-system
> kubectl logs -n kube-system -l k8s-app=kube-dns
> kubectl describe pod -n kube-system <coredns-pod>
> nslookup <service-name> (inside a Pod)
```

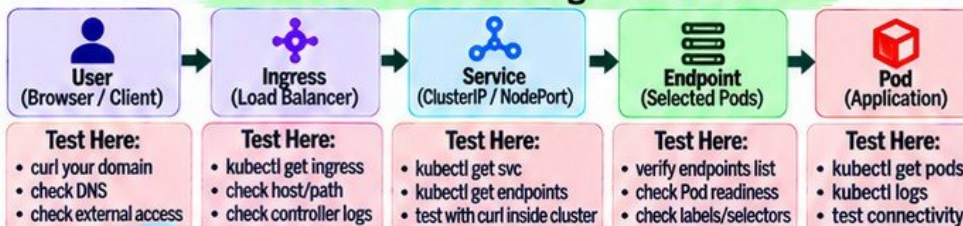
#### How to Fix

- ✓ Ensure CoreDNS Pods are running.
- ✓ Check CoreDNS configuration.
- ✓ Verify the kube-dns Service and endpoints.
- ✓ Review network policies and firewall rules.
- ✓ Check node DNS settings and cluster networking.

#### Junior Tip

Use a temporary debug Pod to test DNS: "kubectl run -it --rm dns-test --image=busybox -- /bin/sh" and then run "nslookup <service>".

## Kubernetes Networking Mental Model



#### KEY CONCEPTS TO REMEMBER

- ✓ Traffic flows: User → Ingress → Service → Endpoint → Pod.
- ✓ Each layer can fail independently.
- ✓ Always check Events and logs.
- ✓ Verify selectors, labels and endpoints.
- ✓ Use temporary debug Pods to test from inside the cluster.
- ✓ DNS is critical for service discovery.
- ✓ Test each layer instead of guessing.

PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

REAL INCIDENTS  
REAL SOLUTIONS  
REAL ENGINEERS  
VERIQTA

# Kubernetes Node and Capacity Outages

## Outages 37 – 39

HEALTHY  
NODES  
HEALTHY  
CLUSTER  
HAPPY  
APPLICATIONS

Understand node failures, resource pressure and capacity issues. Diagnose. Plan. Keep your cluster running.

### 37 Node Becomes NotReady

The node stops communicating with the control plane, so Pods on that node may become unavailable or be rescheduled.

#### Common Symptoms

- Node status shows NotReady.
- Pods on the node become NotReady or are evicted.
- kubelet is not running or unhealthy.
- Network connectivity issues between node and control plane.
- High resource usage (CPU, memory, disk).

#### Diagnose It

```
> kubectl get nodes
> kubectl describe node <node-name>
> kubectl get pods -o wide
> kubectl logs -n kube-system \
  -l k8s-app=kubelet
```

#### How to Fix

- Check node health and kubelet status.
- Verify network connectivity and firewall rules.
- Check system resources (CPU, memory, disk).
- Restart kubelet if needed.
- Replace the node if it cannot be recovered.

#### Junior Tip

Look at the node Conditions and events before taking action. They tell you the real reason.

#### Key Concepts Covered

- CPU and memory resources
- Disk pressure
- Scheduling and the Kubernetes scheduler
- Resource requests and limits
- Node conditions (Ready, MemoryPressure, DiskPressure, PIDPressure)
- Capacity planning and cluster autoscaling

### 38 Resource Pressure Causes Pod Evictions

The node is under resource pressure (CPU, memory or disk), so Kubernetes evicts Pods to protect the node.

#### Common Symptoms

- Pods are evicted with reason Evicted.
- Node conditions show MemoryPressure, DiskPressure or PIDPressure.
- Applications become unavailable or restart frequently.
- High CPU, memory or disk usage on the node.
- Performance degradation.

#### Diagnose It

```
> kubectl describe node <node-name>
> kubectl get events--sort-by=.lastTimestamp
> kubectl top node
> kubectl top pod --all-namespaces
> kubectl describe pod <pod-name>
```

#### How to Fix

- Identify which resource is under pressure.
- Clean up unused resources (images, logs).
- Increase node capacity or scale the cluster.
- Set proper resource requests and limits for workloads.
- Use monitoring and alerts to detect pressure early.

#### Junior Tip

Check the node Events. They show exactly why thePod was evicted (e.g. "evicted due to memory pressure").

### 39 Cluster Has Insufficient Capacity

There are not enough resources in the cluster to schedule new Pods, so they remain in Pending state.

#### Common Symptoms

- Pods remain in Pending state.
- Events show "Insufficient cpu", "Insufficient memory" or "Insufficient nvidia.com/gpu".
- Cluster cannot schedule new workloads.
- Autoscaler is not working or not configured.
- Resource quotas or limits are reached.

#### Diagnose It

```
> kubectl get pods
> kubectl describe pod <pod-name>
> kubectl get nodes
> kubectl describe node <node-name>
> kubectl cluster-info
> kubectl get resourcequota --all-namespaces
```

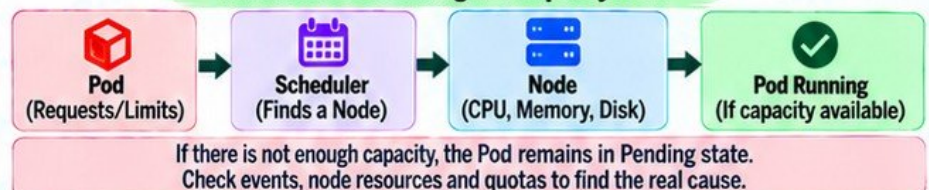
#### How to Fix

- Add more nodes or scale the node group.
- Configure and verify cluster autoscaler.
- Adjust resource requests and limits.
- Optimize resource usage (right-size workloads).
- Review and increase resource quotas if needed.

#### Junior Tip

Look at the scheduler events in the Pod description. They clearly tell you which resource is insufficient.

#### Kubernetes Scheduling and Capacity Flow



PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

REAL INCIDENTS  
REAL SOLUTIONS  
REAL ENGINEERS  
VERIQTA

# Infrastructure as Code Outages

## Outages 40 – 42

**TERRAFORM  
POWERS  
RELIABLE  
INFRASTRUCTURE  
USE IT WISELY  
KEEP SYSTEMS  
RUNNING**

Small changes in code can cause big outages. Plan. Review. Apply carefully. Verify always.

### 40 Terraform Change Accidentally Replaces a Critical Resource

A change in the Terraform code causes a critical resource (e.g. database, load balancer, instance, or network component) to be destroyed and recreated, leading to an outage.

#### Common Symptoms

- Terraform plan shows resource destruction ("-/+").
- Critical service becomes unavailable after apply.
- Resource gets a new ID.
- Data may be lost (e.g. unencrypted EBS, non-replicated database).
- Dependent services fail.

#### Diagnose It

```
> terraform plan
> terraform show
> Check the plan for "destroy" actions
> Review state: terraform state list
> Check resource configuration
```

#### How to Fix

- ✓ Review the plan carefully.
- ✓ Use lifecycle rules (e.g. prevent\_destroy).
- ✓ Use backups and snapshots.
- ✓ Restore from backup if needed.
- ✓ Implement peer review before apply.

#### Junior Tip



Always read the plan output. If you see resources being destroyed, stop and investigate.

#### Key Concepts to Remember

- ✓ Infrastructure as Code (IaC)
- ✓ Terraform plan and apply
- ✓ Destructive changes
- ✓ State management
- ✓ Configuration drift
- ✓ Networking dependencies
- ✓ Change review and approvals
- ✓ Always verify after apply

### 41 Infrastructure Configuration Drift

The real infrastructure (in the cloud) differs from the Terraform code, causing unexpected changes, failed deployments or outages.

#### Common Symptoms

- Terraform plan shows unexpected changes.
- Resources were changed manually in the cloud console.
- Tags, security groups or settings are different.
- Apply tries to modify or replace existing resources.
- Inconsistent environments (dev, staging, prod).

#### Diagnose It

```
> terraform plan
> terraform state show <resource>
> Compare with cloud console
> terraform refresh (legacy)
> Check for manual changes
```

#### How to Fix

- ✓ Identify and document manual changes.
- ✓ Update Terraform code or revert manual changes.
- ✓ Use policies to prevent manual changes (e.g. IAM, SCP, IaC only).
- ✓ Run regular drift detection.
- ✓ Enforce code reviews and audits.

#### Junior Tip



Infrastructure should be managed through code. Avoid making manual changes in the cloud console.

### 42 Bad Infrastructure Change Breaks Networking

A Terraform change modifies or deletes networking components (VPC, subnet, route table, security group, NAT, IGW, etc.), causing loss of connectivity and service outages.

#### Common Symptoms

- Instances or services lose network connectivity.
- Cannot reach the internet or internal services.
- Security group rules are too restrictive.
- Route tables are misconfigured.
- NAT gateway or internet gateway deleted or modified.
- DNS or VPC peering connectivity fails.

#### Diagnose It

```
> terraform plan
> Check networking resources in plan
> Review VPC, subnets, route tables, SGs
> Check flow logs and reachability
> Test connectivity (ping, curl, traceroute)
```

#### How to Fix

- ✓ Review and correct the Terraform code.
- ✓ Restore networking components (if deleted).
- ✓ Verify security group and route table rules.
- ✓ Test connectivity after changes.
- ✓ Use a staging environment for validation.

#### Junior Tip



Networking changes can take down everything. Double-check routes, security groups and dependencies before applying.

#### Terraform Workflow (Production Safe)



**Never treat terraform apply as an ordinary command when the plan contains destructive changes.**

**PLAN SMART  
APPLY SAFE  
SYSTEMS STAY UP**

PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

REAL INCIDENTS  
REAL SOLUTIONS  
REAL ENGINEERS  
VERIQTA

# CI/CD Infrastructure Outages

## Outages 43 – 45

  
AUTOMATE  
DEPLOY  
VERIFY  
ROLLBACK  
KEEP SYSTEMS  
RUNNING

Reliable deployments require reliable infrastructure, access, credentials and verification.

### 43 Deployment Pipeline Cannot Reach Infrastructure

The pipeline cannot reach the target environment (cloud, servers or Kubernetes), so the deployment fails.

#### Common Symptoms

- Connection timeout or refused.
- Cannot reach cloud API endpoints (e.g. AWS, Azure, GCP).
- Network or firewall blocking access.
- Self-hosted runner cannot reach the cluster or server.
- DNS resolution failures.
- VPC, subnet or security group misconfiguration.

#### Diagnose It

```
> ping <host>
> curl -v <endpoint>
> nslookup <domain>
> telnet <host> <port>
> kubectl cluster-info (if using k8s)
> check runner network and firewall rules
```

#### How to Fix

- ✓ Verify network connectivity from runner.
- ✓ Check security groups, firewalls and routes.
- ✓ Confirm VPC, subnet and endpoint access.
- ✓ Test DNS resolution.
- ✓ Ensure the runner has the correct network access (VPN, VPC, peering).

#### Junior Tip

 Test connectivity manually from the runner instead of assuming the network is fine.

#### Key Concepts to Remember

- ✓ Runners (hosted or self-hosted)
- ✓ Credentials and secret management
- ✓ IAM and permissions
- ✓ Network access and connectivity
- ✓ Deployment state and versioning
- ✓ Rollback strategies
- ✓ Verification and monitoring
- ✓ Always test in a safe environment first

### 44 Expired or Invalid Pipeline Credentials

The pipeline uses expired, revoked or incorrect credentials, so it cannot authenticate to the target environment.

#### Common Symptoms

- Authentication failed errors.
- Expired token or access key.
- AccessDenied or Unauthorized errors.
- Cannot assume IAM role.
- Credentials work locally but fail in the pipeline.
- Secrets not found or incorrectly configured.

#### Diagnose It

```
> check pipeline secret configuration
> validate token expiry
> aws sts get-caller-identity
> kubectl auth can-i <action>
> check IAM role and permissions
> review pipeline logs for auth errors
```

#### How to Fix

- ✓ Update expired credentials.
- ✓ Rotate access keys and tokens.
- ✓ Verify IAM role permissions.
- ✓ Ensure secrets are correctly stored in the CI/CD system (GitHub Actions, GitLab CI, Jenkins, etc).
- ✓ Use short-lived credentials and secret management tools.

#### Junior Tip

 Check the exact error message. It usually tells you whether it's an expired token, wrong permissions or the wrong environment.

### 45 Broken Deployment Leaves Production Partially Updated

The deployment fails in the middle, leaving the application in an inconsistent state (some versions old, some new).

#### Common Symptoms

- Some instances running new version and others old.
- Application errors or broken functionality.
- Database migrations applied but application not fully deployed.
- Service disruption or partial outage.
- Rollback fails or is not configured.
- Inconsistent configuration across environments.

#### Diagnose It

```
> check deployment logs
> kubectl get deployments
> kubectl rollout status <deployment>
> check application and database state
> review pipeline history
> verify configuration management state
```

#### How to Fix

- ✓ Identify the current deployment state.
- ✓ Complete the deployment or roll back to a stable version.
- ✓ Use blue/green or canary deployments to reduce risk.
- ✓ Verify application and database state.
- ✓ Implement automated rollback for failed deployments.

#### Junior Tip

 Always verify the service after deployment. Don't assume it worked just because the pipeline finished without errors.

#### CI/CD Deployment Flow



 Never treat a deployment as a normal command. Always review, verify and have a rollback plan.

STABLE  
DEPLOYMENTS  
HAPPY USERS

PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.

REAL INCIDENTS  
REAL SOLUTIONS  
REAL ENGINEERS  
VERIQTA

# Identity, Certificates and Secrets

## Outages 46 – 48

  
SECURE IDENTITIES  
SECURE SYSTEMS  
RELIABLE APPLICATIONS  
KEEP PRODUCTION RUNNING

Authentication, authorization and encryption issues can break production. Understand the real cause.

### 46 IAM Permission Change Breaks Production

A change in IAM permissions (policy, role or user) removes the access needed by the application, so it can no longer access required resources.

#### Common Symptoms

- AccessDenied or Unauthorized errors.
- Application cannot access AWS services (S3, RDS, DynamoDB, etc.).
- Works in one environment but fails in another.
- Sudden failures after an IAM policy change.
- Resources appear to exist but cannot be accessed.

#### Diagnose It

```
> aws sts get-caller-identity
> aws iam simulate-principal-policy
> aws iam get-user
> aws iam list-attached-policies
> check CloudTrail for denied actions
```

#### How to Fix

- ✓ Review recent IAM changes.
- ✓ Use least privilege, not overly restrictive policies.
- ✓ Validate required permissions.
- ✓ Test access with the correct role/user.
- ✓ Monitor and alert on AccessDenied errors.

#### Junior Tip

 Check CloudTrail. It shows exactly which action was denied and why.

#### Key Concepts to Remember

- ✓ Authentication vs Authorization
- ✓ IAM users, roles and policies
- ✓ TLS certificates and certificate chains
- ✓ Secret management and rotation
- ✓ Least privilege access
- ✓ Monitoring and alerting
- ✓ Authentication failures can look like infrastructure failures
- ✓ Always verify identity and permissions first

### 47 TLS Certificate Expires

A TLS certificate expires or becomes invalid, so clients can no longer establish a secure connection to the service.

#### Common Symptoms

- SSL/TLS errors (certificate expired, not yet valid, or unknown issuer).
- Browsers show security warnings.
- Clients cannot connect to the service.
- Services behind load balancers or ingress stop working.
- Sudden outages after certificate expiration date.

#### Diagnose It

```
> openssl s_client -connect example.com:443
> openssl x509 -in cert.pem -text -noout
> check certificate expiration date
> check load balancer / ingress logs
> verify certificate chain and issuer
```

#### How to Fix

- ✓ Renew or replace the expired certificate.
- ✓ Ensure the full certificate chain is installed.
- ✓ Update the certificate in load balancer, ingress or application.
- ✓ Use automated certificate renewal (e.g. Let's Encrypt, AWS ACM).
- ✓ Set expiry monitoring and alerts.

#### Junior Tip

 Set expiry alerts at least 30 days before the certificate expires.

### 48 Secret or Credential Rotation Breaks Applications

A secret, API key or credential is rotated but the application is not updated, so it continues using the old credential and fails to authenticate.

#### Common Symptoms

- Authentication failed errors.
- Invalid API key or token.
- Application suddenly loses access to databases, APIs or services.
- Works in one environment but fails after rotation.
- Intermittent failures if some instances have new credentials and others do not.

#### Diagnose It

```
> check application logs for auth errors
> verify secret version and last rotated time
> check secrets manager (AWS/GCP/Azure)
> validate environment variables
> compare working vs failing instances
```

#### How to Fix

- ✓ Update the application with the new secret.
- ✓ Ensure all instances are updated (e.g. rolling restart).
- ✓ Use a secrets manager (AWS Secrets Manager, Vault, etc.).
- ✓ Implement automated rotation with proper rollout.
- ✓ Test the new credential before rotation.

#### Junior Tip

 Use a deployment process that updates secrets and restarts applications automatically.

#### Dependency Chains



 Authentication failures can look like infrastructure failures. Always check identity, permissions, certificates and secrets before assuming a network, server or application issue.

IDENTITY  
ENABLES  
EVERYTHING

PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.

REAL INCIDENTS  
REAL SOLUTIONS  
REAL ENGINEERS  
VERIQTA

# Observability Failure

## Outages 49 – 50

OBSERVE  
UNDERSTAND  
TROUBLESHOOT  
IMPROVE  
KEEP SYSTEMS  
RUNNING

You cannot fix what you cannot see. Observability turns data into evidence.

49

### Monitoring Fails During the Outage

The service is broken, but dashboards show nothing useful.



#### Common Symptoms

- Dashboards show no errors or incorrect data.
- Metrics stop reporting or show flat lines.
- Logs are missing or incomplete.
- Traces are not available.
- Engineers cannot see the real impact.
- Outage is happening, but monitoring gives a false sense of normal.



#### Diagnose It

- > check if metrics are being scraped
- > verify log ingestion pipeline
- > check exporter and agent status
- > confirm dashboards and data sources
- > test end-to-end telemetry flow



#### How to Fix

- ✓ Restore missing telemetry (metrics, logs, traces).
- ✓ Fix or redeploy broken exporters/agents.
- ✓ Verify data pipelines (e.g. Prometheus, Loki, OpenSearch).
- ✓ Ensure dashboards are using the correct data sources.
- ✓ Test monitoring during and after the incident.



#### Junior Tip



Your monitoring can fail too. Always validate that metrics, logs and alerts are actually working.

50

### Alerting Fails to Detect the Incident

The outage exists long before engineers discover it.



#### Common Symptoms

- No alerts are triggered during the outage.
- Alerts are triggered too late.
- Important alerts are buried in noise.
- Alert thresholds are too high or too low.
- Engineers discover the issue from users, not from monitoring.
- Repeated or irrelevant alerts cause alert fatigue.



#### Diagnose It

- > review alert rules and thresholds
- > check alert history and timelines
- > verify notification channels (email, Slack, PagerDuty)
- > test alerts with real or simulated incidents
- > check for silenced or disabled alerts



#### How to Fix

- ✓ Set meaningful alert thresholds (not too high, not too low).
- ✓ Reduce alert noise and alert fatigue.
- ✓ Ensure critical alerts are enabled and not silenced.
- ✓ Test alerts regularly (game days).
- ✓ Verify notification delivery and escalation.



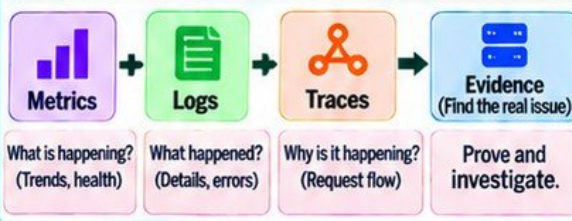
#### Junior Tip



If users tell you before your alerts, your monitoring is not working. Fix the alerts, not just the system.



#### Three Core Signals



#### Common Observability Issues

- Missing telemetry
- Broken exporters or agents
- Incorrect or missing labels
- Bad alert thresholds
- Alert fatigue
- No end-to-end visibility
- Blind spots in distributed systems
- Not monitoring the monitoring system



#### Best Practices

- ✓ Instrument all critical services
- ✓ Monitor metrics, logs and traces
- ✓ Set meaningful and actionable alerts
- ✓ Reduce alert noise
- ✓ Create dashboards for key services
- ✓ Test alerts regularly
- ✓ Monitor the monitoring system
- ✓ Use synthetic checks for critical paths

PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

REAL INCIDENTS  
REAL SOLUTIONS  
REAL ENGINEERS  
VERIQTA

# Production Outage Troubleshooting Playbook

A REUSABLE METHOD FOR ALL 50 INCIDENTS

Stay calm. Be methodical. Find the real cause. Fix it. Learn from it.

INVESTIGATE  
SOLVE  
LEARN  
IMPROVE  
STRONGER  
SYSTEMS  
VERIQTA

## 1 Confirm the Symptom 🔍

What exactly is failing?

- Identify the error, behavior or impact.
- Check user reports, logs and monitoring.
- Understand what is not working (e.g. error messages, timeouts, slow responses, wrong data).
- Avoid assumptions.

 Be specific about the problem.

## 2 Establish Scope 📍

One user, one host, one region, or everyone?

- Is it a single user or all users?
- Is it limited to one service, host, zone or region?
- Check if it is environment-specific (dev, staging, prod).
- Determine the business impact.

 The scope tells you how serious and where to investigate.

## 3 Ask What Changed 🔄

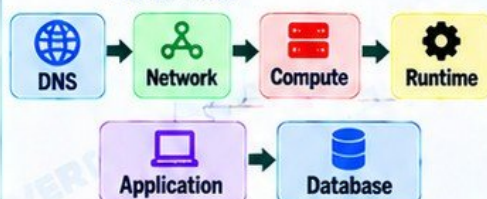
Deployment, infrastructure, configuration, credentials, DNS, certificate?

- Check recent deployments.
- Review infrastructure changes (e.g. Terraform, cloud console).
- Look for configuration updates.
- Check for credential or permission changes.
- Verify DNS, TLS certificates and network changes.

 Recent changes often explain the cause.

## 4 Trace Dependencies 📊

Follow the request through the full stack.



 Find where in the chain the failure occurs.

## 5 Gather Evidence 📄

Metrics, logs, events, system state, configuration.

- Check metrics (CPU, memory, latency, error rates).
- Review logs (application, system, cloud, audit).
- Look at events (Kubernetes, cloud provider, CI/CD).
- Inspect current system state and configuration.
- Gather evidence before making changes.

 Evidence leads to the truth.

## 6 Form and Test a Hypothesis 💡

Do not make random changes.

- Based on evidence, form a possible cause.
- Test the hypothesis with safe, reversible checks.
- Validate or reject it with data.
- Avoid making multiple changes at once.
- If the hypothesis is wrong, adjust and try again.

 Think, don't guess.

## 7 Apply the Smallest Safe Fix 🛠️

Reduce additional blast radius.

- Choose the least risky fix.
- Make targeted changes.
- Consider rollback options.
- Avoid large, untested changes during an outage.
- Communicate the fix to the team.

 Fix safely and limit the impact.

## 8 Verify Recovery ✅

Never assume the fix worked.

- Check that the original issue is resolved.
- Verify all related functionality.
- Monitor metrics, logs and user reports.
- Confirm there are no side effects.
- Get validation from real users if possible.

 Prove it's fixed.

## 9 Monitor 📈

Watch for recurrence.

- Keep monitoring metrics, logs and alerts.
- Look for related errors or partial failures.
- Ensure the system is stable over time.
- Be alert for signs of recurrence.
- Do not close the incident too early.

 Stability over time confirms recovery.

## 10 Document and Prevent 📄

Turn the incident into engineering knowledge.

- Write a clear incident summary.
- Include timeline, root cause and what was fixed.
- Identify what can be improved (monitoring, alerts, processes, automation).
- Create action items and assign owners.
- Update runbooks and share knowledge with the team.

 Every incident should make you stronger.

## Key Takeaways

- Use a structured process.
- Follow evidence, not assumptions.
- Understand the full system and dependencies.
- Make safe, minimal changes.
- Verify and monitor.
- Document and prevent.
- Turn outages into learning opportunities.

## Remember

- Calm engineers solve problems faster.
- Most outages are caused by change.
- Observability is your best friend.
- Small fixes prevent bigger disasters.
- Good documentation prevents repeat incidents.
- Reliability is a continuous process, not a one-time fix.

PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

REAL INCIDENTS  
REAL SOLUTIONS  
REAL ENGINEERS  
VERIQTA

# Final Production Challenge

## The Outage Is Yours

A REALISTIC INCIDENT. YOUR INVESTIGATION. YOUR SOLUTION.

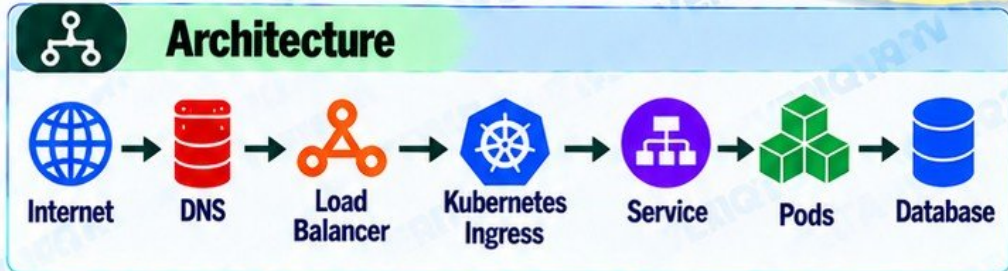
Think like an engineer. Follow a method. Find the root cause.

  
PRACTICE  
INVESTIGATE  
TROUBLESHOOT  
LEARN  
IMPROVE  
BE A STRONGER  
ENGINEER  
VERIQTA

### Scenario

Customers report:  
**"The website is down."**

You are the on-call engineer.  
Investigate, find the cause,  
and restore the service.



- ### Observed Clues
- 1 DNS resolves successfully.
  - 2 Load balancer responds.
  - 3 HTTP requests return 503.
  - 4 Several Pods are restarting.
  - 5 One Kubernetes node reports memory pressure.
  - 6 Database connections are unusually high.
  - 7 A deployment occurred 15 minutes earlier.

- ### Your Investigation
- 1 What should you investigate first?
  - 2 What evidence do you need?
  - 3 Which commands should you run?
  - 4 What changed?
  - 5 What is the likely blast radius?
  - 6 What is the safest recovery action?
  - 7 How will you verify recovery?
  - 8 How could this incident be prevented?
- "REAL ENGINEERS SOLVE PROBLEMS."**  
VERIQTA

### Useful Commands (Examples)

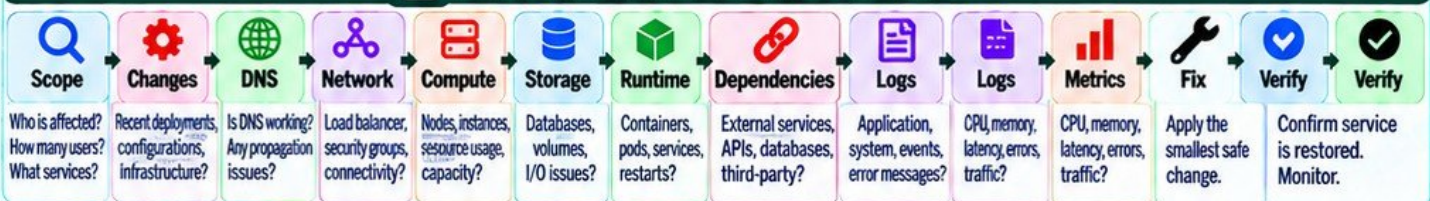
```

> kubectl get pods -A
> kubectl describe pod <pod-name>
> kubectl logs <pod-name>
> kubectl get nodes
> kubectl describe node <node-name>
> kubectl top pods
> kubectl top nodes
> kubectl get events -A
> kubectl rollout history deployment/<name>
  
```

- ### Key Areas to Investigate
- ✓ Recent deployment changes
  - ✓ Pod restarts and logs
  - ✓ Node resource usage (CPU, memory)
  - ✓ Application errors (503)
  - ✓ Database connection spikes
  - ✓ Service and ingress configuration
  - ✓ Resource requests and limits
  - ✓ Events and system messages

- ### What a Good Answer Includes
- ✓ Clear starting point and reasoning
  - ✓ Specific commands and evidence
  - ✓ Identification of the change
  - ✓ Likely root cause with explanation
  - ✓ Blast radius assessment
  - ✓ Safe recovery plan
  - ✓ Verification steps
  - ✓ Long-term prevention measures

### Junior Infrastructure Outage Checklist



PRACTICE  
TROUBLESHOOT  
IMPROVE  
REPEAT

REAL PROBLEMS. REAL SOLUTIONS. A STRONGER YOU.